Capstone Project Report

Compressing LLMs with Holograms

Karapet Khachatryan
karapet_khachatryan@edu.aua.am

Supervisor: Suren Khachatryan

American University of Armenia, Yerevan, Armenia
Bachelor of Science in Computer Science

1 Abstract

This capstone project investigates whether holography-inspired encoding and decoding can be used as an exploratory compression mechanism for large language model (LLM) weight matrices. The central idea is to treat neural-network weights as spatial data: matrix indices define coordinates, while values define height, depth, intensity, or phase. The project first validates the idea on controlled VTP point-cloud objects, then connects the same pipeline to matrix-based representations of model weights. The implemented workflow includes normalization, matrix-to-geometry conversion, VTP point-cloud generation, multi-view hologram-like encoding, separate amplitude and phase storage, confidence-volume decoding, and reconstruction evaluation using file size, RMSE, MAE, F1 score, voxel IoU, and related metrics. The experiments show that the method can encode and reconstruct recognizable geometric structure and can achieve moderate compression in a cube experiment, but strict numerical accuracy is still limited, especially for matrix roundtrip reconstruction. The project therefore presents holographic encoding as a research direction rather than a finished replacement for established LLM compression methods. Future work should focus on learned decoders, such as U-Net-style models, to improve reconstruction from amplitude-and-phase hologram packages.

2 Introduction

Large language models have become central to modern artificial intelligence, but their practical use is constrained by parameter size, memory requirements, storage cost, and inference cost. Transformer-based models rely heavily on repeated matrix operations, and architectures such as BERT show how stacked attention and feed-forward layers depend on many large weight matrices [8, 9]. As models scale, storing and transferring these matrices becomes increasingly expensive. For this reason, model compression has become an important research area, with common approaches including quantization, pruning, knowledge distillation, and low-rank approximation [10].

This capstone approaches the compression problem from a different direction. Instead of only asking how many parameters can be removed or how many bits should represent each weight, it asks whether neural weights can be transformed into another representational domain before storage. The inspiration is holography. Gabor's original holographic principle showed that information about a wavefront can be recorded indirectly through an interference pattern rather than through an ordinary intensity image [1]. In physical and digital holography, the recorded pattern may not visually resemble the object, but it can still support reconstruction of object information under suitable optical or computational conditions [2]. This suggests a broader representation idea: information does not always need to be stored in the same form in which it is later used.

The project applies this idea to model weights. A weight matrix can be interpreted as a surface in which row and column indices define spatial coordinates and the weight value defines height, depth, intensity, or phase. Multiple matrices can also be packed into a larger spatial representation, stacked into a volume, or mapped onto curved surfaces such as hemispheres. After this conversion, the resulting object can be encoded into a hologram-like two-dimensional representation and then decoded back into an approximate point cloud or matrix.

The research does not claim that holographic encoding is currently better than standard compression methods. Its goal is narrower: to test whether a hologram-inspired representation can preserve meaningful numerical or geometric structure while reducing storage size. Early experiments show that reconstruction is sensitive to hologram resolution, depth handling, view selection, normalization, and whether amplitude and phase are both stored. Simple objects and low-variation matrices are easier to reconstruct, while complex matrices expose the current decoder's limitations. These limitations are valuable because they show what information the pipeline loses and what future decoders must recover.

The main goal of the capstone is therefore to design, implement, and evaluate a hologram-inspired encoding–decoding pipeline for neural weight matrices. The project uses simple geometric objects, such as cubes and bell-shaped surfaces, as controlled test cases before applying similar ideas to language-model matrices. Success is evaluated through reconstruction metrics, visual inspection, file-size comparison, and analysis of whether depth, phase, and matrix structure are preserved.

3 Literature Review

Large language models are built mainly from numerical weight matrices, especially inside Transformer architectures. The Transformer introduced self-attention as a highly parallelizable replacement for recurrent sequence modeling, and BERT later demonstrated the effectiveness of stacked Transformer layers for language understanding [8, 9]. These architectures motivate compression because high performance is often tied to storing and processing many large matrices.

Most existing LLM compression research follows several major directions: quantization, pruning, knowledge distillation, and low-rank decomposition [10]. Quantization reduces the number of bits used to represent weights or activations. GPTQ is an example of post-training quantization that uses approximate second-order information for low-bit GPT-style models [11]. SmoothQuant addresses activation outliers by shifting part of the quantization difficulty from activations to weights, enabling efficient weight-and-activation quantization [12]. AWQ similarly argues that only a small fraction of weights are especially important and protects salient weight channels using activation-aware scaling [13].

Pruning removes less important parameters or structures from the model. SparseGPT, for example, shows that large GPT-family models can be pruned in one shot while preserving much of their performance [14]. Knowledge distillation trains a smaller model to imitate a larger model; DistilBERT is a classic example that reduces BERT's size while retaining much of its language understanding ability [15]. Low-rank methods are relevant because weight matrices and model updates often contain lower-dimensional structure. LoRA, although mainly used for parameter-efficient fine-tuning, shows that Transformer updates can be represented through low-rank matrices with far fewer trainable parameters [16].

These methods compress models mostly inside the original neural-network representation. Quantization keeps the matrix layout but lowers precision; pruning removes weights; distillation transfers behavior into a smaller model; and low-rank factorization approximates matrices with simpler components. This capstone explores a different question: whether LLM parameters can be treated as spatial signals and encoded through a holography-inspired representation. In this view, each weight matrix is not only a table of values but also a surface or layer. A stack or packing of such layers can be interpreted as a three-dimensional object, encoded into a hologram-like pattern, and decoded back into an approximate reconstruction.

The theoretical basis comes from holography. Gabor introduced holography as a method for recording and reconstructing wavefront information [1]. Unlike ordinary images, which store intensity from a viewpoint, holograms encode information about a wavefield through interference. Digital and computer-generated holography build on this idea computationally; in many cases the data is complex-valued and phase information is essential for reconstruction [2]. This matters for the capstone because the project is not ordinary image compression. It asks whether a lower-dimensional encoded representation can preserve enough information to reconstruct a higher-dimensional structure.

Hologram compression is itself a specialized problem. JPEG Pleno Holography was developed because holographic data has signal properties that differ from ordinary natural images, including complex-valued information and high-frequency interference patterns [3]. Recent work also applies neural networks to hologram compression. Shi et al. proposed neural compression for complex-valued hologram images and videos, showing that learned methods can operate directly on holographic data rather than treating it as a normal image [6]. Reviews of deep learning for computer-generated holography also describe data-driven, physics-driven, and jointly optimized methods, suggesting that neural models can support both hologram generation and reconstruction [7].

Compressive holography is especially relevant because it studies reconstruction of three-dimensional information from fewer measurements. Brady et al. describe compressive holography as a way to reconstruct multidimensional images from lower-dimensional holographic data by using compressed-sensing assumptions [4]. This connects to the capstone hypothesis: if model-weight structures contain redundancy, a compressed holographic representation might preserve useful structure even when the encoded file is smaller than the original matrix or VTP representation.

The literature also shows why the problem is difficult. Birdi et al. note that replay or back-propagation in digital holography is a Hermitian transpose rather than a true inverse of hologram formation, so naive reconstruction only approximates the original three-dimensional object [5]. This directly explains a challenge in the capstone experiments:

a decoded point cloud or matrix may visually resemble the original while still having high numerical error. For that reason, evaluation must include strict metrics such as RMSE, MAE, cosine similarity, nearest-neighbor distances, voxel IoU, compression ratio, and eventually downstream model-performance tests.

Overall, the literature supports the project as an exploratory but motivated direction. LLM compression research shows that neural models often contain redundancy; holography and compressive imaging show that spatial information can be encoded indirectly; and neural hologram compression suggests that learned decoders may improve reconstruction when classical decoding is insufficient. The research gap is that these ideas are rarely applied directly to LLM weight matrices. This capstone contributes by testing whether holography-inspired encoding can serve as an alternative compression framework for neural-network parameters, beginning with simple three-dimensional objects and moving toward matrix reconstruction.

4 Problem Formulation

LLMs require large amounts of storage because their behavior is represented through millions or billions of numerical parameters. In Transformer models, many of these parameters are stored as weight matrices inside attention and feed-forward layers. Existing compression methods reduce size by changing precision, sparsity, architecture, or factorization, but they usually keep the model within the traditional matrix-storage framework. This project formulates a different research problem: whether model parameters can first be represented as a spatial object and then compressed through a holography-inspired encoding process.

The central analogy is between holography and neural-weight storage. In holography, information about a three-dimensional object can be recorded on a two-dimensional hologram-like surface and later reconstructed under appropriate conditions. Similarly, an LLM can be viewed as a layered numerical structure: each weight matrix is a two-dimensional surface, and a sequence or grouping of matrices can be interpreted as a three-dimensional object. The problem is whether this spatial representation of model parameters can be encoded into a smaller hologram-like package and reconstructed with acceptable accuracy.

Formally, let a model contain a set of weight matrices:

$$W = \{W_1, W_2, \ldots, W_n\},$$

where each W_i represents a matrix from one part of the model, such as an attention layer or feed-forward layer. These matrices are transformed into a spatial representation P, such as a point cloud, surface, or volume, where matrix indices define spatial coordinates and matrix values define height, depth, intensity, phase, or density. The encoder then constructs a hologram-like representation:

$$E(P) = H,$$

where H is stored as one or more amplitude and phase images plus reconstruction metadata. A decoder then estimates the original spatial object:

$$D(H) = \hat{P},$$

and the reconstructed object is mapped back into approximate matrices:

$$\hat{W} = \{\hat{W}_1, \hat{W}_2, \ldots, \hat{W}_n\}.$$

The main research problem is to determine whether $\hat{W}$ remains sufficiently close to W while the encoded representation H is smaller than the original storage format. This creates a compression–reconstruction trade-off. A useful method must reduce file size while preserving numerical and structural information. This is difficult because holographic encoding may lose depth information, phase detail, high-frequency structure, or small numerical variations. For ordinary objects, a visually similar reconstruction may be acceptable; for neural weights, small numerical errors may affect model behavior. The evaluation therefore includes visual inspection, file-size comparison, RMSE, MAE, cosine similarity, nearest-neighbor distance, voxel IoU, F1 score, and, in future work, downstream model-performance tests.

The project begins with simple geometric objects, such as cubes and bell-shaped surfaces, because they provide controlled cases where reconstruction quality can be interpreted visually and numerically. After that, the framework is applied to matrix data that resembles or comes from real LLM layers. This staged structure separates the basic holographic reconstruction problem from the more difficult question of whether reconstructed neural-network weights preserve model behavior.

The main research question is:

Can a holography-inspired two-dimensional encoding preserve enough information from spatial representations of LLM weight matrices to provide meaningful compression with acceptable reconstruction accuracy?

The project addresses five subproblems:

1. **Representation:** How should weight matrices be converted into spatial objects suitable for holographic encoding?
2. **Encoding:** What hologram-like encoding can store the spatial representation compactly?
3. **Decoding:** How accurately can the original object or matrix be reconstructed from the encoded package?
4. **Evaluation:** Which metrics best measure reconstruction quality for compression purposes?
5. **Trade-off:** How much file-size reduction is possible before reconstruction quality becomes unacceptable?

The expected outcome is not to prove that holography is already a practical replacement for established LLM compression methods. Instead, the goal is to test whether this alternative representation has research potential. Strong reconstruction would suggest a new direction for neural-network storage; weak reconstruction still clarifies the limitations of applying holographic principles to LLM compression and identifies what improvements are needed, such as phase-aware encoding, multi-view holograms, better depth recovery, and learned decoders.

5 Proposed Methodology

The methodology follows a staged experimental design. The project does not begin by compressing a full LLM directly. Instead, it first validates the holography-inspired encoding and decoding pipeline on controlled three-dimensional VTP point-cloud objects. This makes errors easier to diagnose: if the method cannot preserve the geometry of simple objects, then applying it to sensitive neural-network parameters would not be meaningful.

The first stage is geometric validation. A VTP file represents a three-dimensional object as a set of spatial points. The pipeline reads these points, normalizes their coordinates into a common range, and processes them using consistent holographic parameters such as hologram size, depth range, spatial extent, wavelength, and pixel pitch. Normalization is required because input objects may have different scales and coordinate systems.

After normalization, the object is encoded from one or more virtual viewpoints. The implementation supports single-view, six-view cube, and circular multi-view configurations. The six-view configuration is especially important for closed objects because it observes the object from front, back, left, right, top, and bottom directions. This reduces the depth ambiguity that would occur if the object were represented by only one projection.

For each view, the point cloud is rotated into the view coordinate system and divided into depth bins. Points in each depth slice are projected onto a two-dimensional complex plane, assigned phase-aware values, and propagated using an angular-spectrum-style propagation step. The propagated slices are summed to form a complex hologram field. The encoded output for each view is stored as an amplitude image, a phase image, and metadata. The metadata contains information needed for reconstruction, including object bounds, normalization parameters, view rotations, depth-bin count, hologram size, and optical settings.

The second stage is decoding. The decoder reconstructs the complex field from the stored amplitude and phase images. For each view and depth bin, the field is back-propagated to estimate where object evidence may exist. Instead of directly choosing the brightest pixels as final points, the newer decoder maps candidate evidence into a shared three-dimensional confidence volume. Voxels supported by multiple views are treated as more reliable than voxels supported by only one view. The decoder can then apply minimum view-support filtering, confidence thresholding, and connected-component filtering before converting the surviving voxels into a reconstructed VTP point cloud.

The third stage is evaluation. Reconstruction quality is measured visually and numerically. The reporting module compares the original and reconstructed VTP files using point counts, file sizes, compression ratio, nearest-neighbor RMSE in both directions, percentile errors, symmetric Hausdorff distance, precision, recall, F1 score under several thresholds, voxel Intersection over Union, and cube-surface metrics such as outside-bounding-box percentage. Matrix experiments are evaluated using numerical matrix similarity measures such as RMSE, MAE, and relative similarity.

The final stage connects the geometric pipeline to LLM weights. A weight matrix is interpreted as a spatial surface where row and column indices define coordinates and the weight value defines height, depth, intensity, phase, or density. Multiple matrices can be packed, stacked, or curved into a three-dimensional representation. The same holography-inspired encoder can then compress this representation into amplitude and phase images, and the decoder

can attempt to reconstruct approximate matrices. This staged methodology keeps the project focused: simple objects test whether the encoding–decoding mechanism works at all, while matrix experiments test whether the mechanism has potential as a compression tool for neural-network parameters.

6 Framework Design and Parameter Exploration

After defining the general methodology, the project implements an experimental framework for testing holography-inspired compression and reconstruction. The framework is designed for repeated experiments rather than a single fixed run. Its purpose is to explore how encoding and decoding settings affect compression ratio, reconstruction accuracy, and geometric stability. Because this approach is experimental, there is no known optimal configuration for converting a three-dimensional object, or later a set of neural-network weight matrices, into a hologram-like representation.

The framework is organized as a full pipeline that takes an input VTP point cloud, encodes it into a hologram package, decodes the package back into a reconstructed VTP point cloud, and then generates strict evaluation reports. The implementation supports three main research activities: encoding, decoding, and quantitative comparison. In addition, an automated `run-all` mode connects all stages into one experiment, making it possible to repeatedly test different parameter configurations.

6.1 Framework Overview

The implemented framework follows the structure:

$$\text{Input VTP object} \rightarrow \text{Holographic encoding} \rightarrow \text{Amplitude/phase image package} \rightarrow$$

$$\rightarrow \text{Confidence-volume decoding} \rightarrow \text{Reconstructed VTP object} \rightarrow \text{Strict metric report}$$

The input object is represented as a VTP point cloud. This format is useful for the first stage of the project because it allows simple three-dimensional objects, such as cubes or bell-shaped surfaces, to be represented as sets of spatial points. The framework first reads the point cloud, normalizes its coordinates, and prepares it for holographic encoding. Normalization is necessary because different objects may have different physical sizes or coordinate ranges, while the holographic encoder expects the object to fit within a controlled spatial extent.

The encoded representation is stored as a collection of image files and metadata. For each view, the framework saves one amplitude image and one phase image. The metadata file stores the information needed for reconstruction, including the original point count, original bounds, normalization center and scale, hologram size, number of depth bins, optical parameters, view mode, and view rotation matrices. This design reflects the idea that a hologram should not only store intensity-like information, but also phase information, since phase plays an important role in wave-based reconstruction.

6.2 Encoding Module

The encoding module is responsible for transforming a normalized three-dimensional point cloud into a hologram-like representation. First, the object is observed from a selected virtual view. The framework supports multiple view configurations, such as a single front view, a six-direction cube view, and circular multi-view settings. The six-view configuration is especially useful for three-dimensional reconstruction because it captures information from the front, back, left, right, top, and bottom directions.

For each view, the object points are rotated into the view coordinate system. The depth axis is then divided into a fixed number of depth bins. Each depth bin represents a slice of the object at a certain distance from the hologram plane. Points in each depth slice are projected onto a two-dimensional complex plane. Random phase values are assigned to the projected points, and the slice is propagated using an angular spectrum propagation method. The propagated fields from all depth slices are then summed to form the final complex hologram field for that view.

The encoding process can be summarized as:

$$P \rightarrow P_v \rightarrow \{P_{v,1}, P_{v,2}, \ldots, P_{v,d}\} \rightarrow \{F_{v,1}, F_{v,2}, \ldots, F_{v,d}\} \rightarrow H_v,$$

where P is the normalized point cloud, P_v is the point cloud rotated into a specific view, $P_{v,i}$ is the set of points belonging to depth slice i, $F_{v,i}$ is the propagated complex field for that slice, and H_v is the final hologram field for view v.

The framework provides several encoding parameters:

- `holo-size`: controls the resolution of the generated hologram images.
- `depth-bins`: controls how many depth slices are used to represent the object.
- `views`: controls the virtual camera configuration, such as `single`, `cube6`, or circular views.
- `wavelength`: controls the simulated wavelength used in wave propagation.
- `pixel-pitch`: controls the physical spacing between hologram pixels.
- `extent`: controls the spatial range covered by the hologram plane.
- `z-base` and `z-span`: define the physical depth range used during propagation.
- `splat-sigma`: controls the smoothing or spreading of projected points on the hologram plane.

These parameters allow the framework to test different storage-quality trade-offs. For example, a smaller `holo-size` may improve compression because the output images become smaller, but it may also lose fine spatial details. A larger number of `depth-bins` may preserve depth more accurately, but it also increases the complexity of the encoding and decoding process. Similarly, using more views may improve three-dimensional reconstruction, but it increases the number of stored amplitude and phase images.

6.3 View Configuration

A major capability of the framework is the ability to encode the object from different view configurations. The simplest configuration is a single view, where the object is observed from one direction. This produces the smallest encoded package, but it also loses the most depth information. For simple surfaces, a single view may be enough, but for closed three-dimensional shapes such as cubes, one view cannot fully describe all sides of the object.

The `cube6` setting uses six orthogonal views: front, back, right, left, top, and bottom. This is the most important view mode for the early experiments because it provides multi-directional evidence about the object. In holographic decoding, this helps separate true object structure from artifacts that may appear in only one projection. The framework also supports circular view modes, such as `circle8` or `circle12`, where views are distributed around the object. These settings can be useful for testing whether reconstruction improves when the object is observed from many angles.

This view system is especially relevant for the later LLM-compression stage. If weight matrices are converted into three-dimensional structures, different views may capture different aspects of the matrix geometry. Therefore, the framework allows experiments not only with compression level, but also with the amount of geometric information given to the decoder.

6.4 Decoding Module

The decoding module reconstructs a point cloud from the stored amplitude and phase images. First, the amplitude and phase images are loaded and combined to recover the complex hologram field. Then, for each view and each depth bin, the framework performs backward propagation. This produces an estimated intensity map for that depth. Instead of directly converting all bright pixels into points, the framework selects only the strongest candidate pixels according to a threshold and top-k rule.

The most important feature of the decoder is the confidence-volume approach. The decoder creates a shared three-dimensional voxel grid. Candidate points from each back-propagated view are mapped into this grid. Each view contributes confidence values to the voxels it supports. The result is a three-dimensional confidence volume:

$$C(x, y, z),$$

where higher values indicate stronger evidence that a voxel belongs to the original object.

This design improves reconstruction because it does not trust a single view independently. A voxel is considered more reliable when it is supported by multiple views. The decoder can require a minimum number of supporting views, remove weak-confidence voxels, and apply connected-component filtering to remove isolated noise. The reconstructed point cloud is extracted only from the surviving high-confidence regions.

The decoding parameters include:

- `volume-resolution`: defines the size of the three-dimensional confidence grid.

- `min-view-support`: requires a voxel to be supported by at least a certain number of views.
- `support-radius`: expands per-view support to nearby voxels.
- `threshold-percentile`: keeps only strong pixels in each back-propagated depth slice.
- `volume-confidence-percentile`: removes weak voxels after multi-view fusion.
- `decode-oversample`: controls the number of candidate points considered before volume fusion.
- `top-k-per-slice`: manually controls how many strong pixels are selected per depth slice.
- `decoder-smooth-sigma`: optionally smooths the reconstructed intensity maps.
- `clip-extent`: defines the spatial range of the confidence volume.
- `confidence-gamma`: controls how strongly confidence values influence point selection.
- `jitter-fraction`: optionally adds small random displacement within selected voxels.
- `force-target-points`: forces the reconstructed point cloud to match the original point count.
- `max-output-points`: caps the number of reconstructed points.
- `min-output-points`: sets a minimum number of points for visualization.
- `connected-filter`: keeps the largest connected confidence components.

These settings allow detailed exploration of the reconstruction process. For example, increasing `min-view-support` may reduce noise because reconstructed points must be confirmed by multiple views. However, if this value is too high, some real points may be removed. Increasing `volume-resolution` gives a finer reconstruction grid, but also increases computational cost. A high `threshold-percentile` keeps only the brightest evidence, which may improve precision but reduce recall. Connected filtering can remove isolated artifacts, but it may also remove small valid components if the object has disconnected structures.

6.5 Quality-First Reconstruction

The framework uses a quality-first reconstruction strategy. By default, it does not force the reconstructed point cloud to contain exactly the same number of points as the original object. This is important because forcing the same point count can create artificial points in locations where the decoder has weak evidence. Such points may improve visual density but damage strict geometric metrics.

Instead, the decoder outputs points only from confident surviving voxels. If the confidence volume contains fewer reliable voxels, the reconstructed object will contain fewer points. This makes the evaluation more honest because the reconstruction reflects what the hologram package actually preserved. Exact point-count reconstruction is still supported through the `force-target-points` setting, but it is treated as an optional mode rather than the default.

This distinction is important for the capstone project. In model compression, artificially reconstructing extra parameters without reliable information may lead to numerical noise. The quality-first setting is more aligned with the long-term goal of preserving meaningful structure rather than simply matching the original file size or point count.

6.6 Automated Experiment Mode

The framework also includes an automated experiment mode. In this mode, the user provides an input VTP file and an experiment directory. The framework then performs encoding, decoding, and reporting automatically. The output directory contains the encoded hologram package, the reconstructed VTP file, and strict metric reports.

The automated experiment structure is:

$$\texttt{input.vtp} \rightarrow \texttt{encoded\_hologram/} \rightarrow \texttt{reconstructed.vtp} \rightarrow \texttt{report/strict\_metrics}.$$

This mode is especially useful for parameter exploration. By changing parameters such as `holo-size`, `depth-bins`, `volume-resolution`, or `min-view-support`, the same object can be tested under many different compression and reconstruction conditions. Each experiment produces comparable output files and metrics, allowing the project to identify which settings produce the best balance between compression ratio and reconstruction accuracy.

6.7 Evaluation and Reporting Capability

The framework includes a strict reporting module that compares the original and reconstructed point clouds. The report includes point counts, file sizes, compression ratio, nearest-neighbor reconstruction errors, symmetric Hausdorff distance, precision, recall, F1 score, voxel Intersection over Union, and optional cube-surface metrics.

The nearest-neighbor metrics are computed in two directions. The original-to-reconstructed distance measures how well the reconstructed object covers the original object. The reconstructed-to-original distance measures how much of the reconstructed object lies close to the original. This distinction is important because a reconstruction may cover the original well while also producing many extra noisy points, or it may be precise but incomplete.

The framework also computes precision, recall, and F1 score under several distance thresholds. These metrics are useful because they convert geometric distance into an interpretable quality measure. Voxel IoU measures the overlap between occupied spatial cells in the original and reconstructed objects. For cube-like objects, additional metrics measure how many reconstructed points fall outside the original bounding box and how close the reconstructed points are to the nearest cube face.

6.8 Parameter Exploration Strategy

The large number of available parameters makes the framework suitable for systematic experimentation. The main experimental strategy is to vary one group of parameters at a time while observing both compression and reconstruction quality.

The first group is the storage-related encoding parameters. These include `holo-size`, `depth-bins`, and `views`. Reducing `holo-size` usually improves compression, because the amplitude and phase images become smaller. However, it may also reduce the spatial resolution of the encoded object. Reducing the number of views also decreases storage cost, but it may weaken depth reconstruction. Increasing `depth-bins` may improve depth representation, but it can make decoding more computationally expensive.

The second group contains reconstruction filters such as minimum view support, threshold percentile, volume confidence percentile, and connected-component filtering. These parameters control how strict the decoder is. A stricter decoder removes more noise, but it may also remove valid parts of the object. A more permissive decoder keeps more points, but it may introduce artifacts.

The third group is the output-control parameters. These include `force-target-points`, `max-output-points`, `min-output-points`, and `jitter-fraction`. These parameters affect the final reconstructed point cloud rather than the hologram itself. They are useful for testing whether visual quality, point-count matching, or strict numerical accuracy should be prioritized.

Overall, the framework supports a controlled investigation of the compression–reconstruction trade-off. It allows the project to answer questions such as:

- How much can the hologram size be reduced before reconstruction quality collapses?
- How many views are necessary for stable three-dimensional reconstruction?
- Does phase storage improve reconstruction compared with amplitude-only storage?
- Does multi-view confidence filtering reduce noise compared with direct peak extraction?
- Which settings produce the best balance between compression ratio and geometric accuracy?

6.9 Connection to LLM Weight Compression

Although the current framework operates on VTP point clouds, it is designed as a preparation stage for LLM weight compression. The first goal is to test whether simple three-dimensional structures can be compressed and reconstructed using a holography-inspired representation. Once this is established, neural network weight matrices can be converted into similar spatial structures.

A weight matrix can be represented as a surface where the row index corresponds to one spatial axis, the column index corresponds to another spatial axis, and the weight value corresponds to height, depth, density, or intensity. Multiple matrices can be stacked or packed together to form a three-dimensional object. The same encoding pipeline can then be applied:

$$\begin{aligned}\text{Weight matrices} &\to \text{3D spatial representation} \to \text{Amplitude/phase package} \\ &\to \text{Reconstructed representation} \to \text{Approximate weight matrices.}\end{aligned}$$

Thus, the framework is not only a tool for reconstructing cubes or synthetic shapes. It is a testing environment for studying whether holographic principles can preserve structured numerical data under compression. The experiments with simple objects provide the necessary foundation before moving to more complex and sensitive data, such as LLM parameters.

7 Spatial Representation of Weight Matrices

Before applying the holography-inspired encoding process, the numerical data must be converted into a spatial format. This step is necessary because the implemented framework operates on VTP point-cloud objects rather than directly on raw matrix files. One important methodological question in this project is how a collection of weight matrices should be represented as a three-dimensional object.

This representation stage is important because the quality of reconstruction depends not only on the holographic encoder and decoder, but also on how the original numerical data is transformed into geometry. If the matrix-to-VTP conversion loses important numerical structure, then even a strong holographic decoder cannot fully recover the original information. For this reason, several representation strategies were considered during the project.

7.1 Normalization as a Common Preprocessing Step

Normalization is applied before converting matrices into spatial objects. Neural network weight matrices may contain very small, very large, positive, and negative values. If these values are used directly as spatial coordinates, the resulting object may become distorted or difficult to encode. All matrix values are normalized into the interval $[-1, 1]$.

The normalization formula is:

$$W'_{ij} = 2 \cdot \frac{W_{ij} - W_{\min}}{W_{\max} - W_{\min}} - 1,$$

where W_{ij} is the original matrix value, $W_{\min}$ is the minimum value in the matrix or matrix collection, and $W_{\max}$ is the maximum value. This transformation maps the smallest value to -1, the largest value to 1, and all intermediate values into the same controlled range.

This normalization is useful for three-dimensional representation because positive and negative values can be represented as positions above or below a central reference level. It also prevents a small number of large values from dominating the geometry. In this project, normalization to $[-1, 1]$ is treated as the common preprocessing step for all matrix-to-VTP representation strategies.

7.2 Single Large Square Matrix Representation

One representation strategy is to combine all selected weight matrices into one large square matrix before converting them into a VTP object. In this approach, the original matrices are placed together inside a larger two-dimensional canvas. If the model contains matrices

$$W = \{W_1, W_2, \ldots, W_n\},$$

then these matrices are arranged into one combined matrix:

$$W_{\text{big}} = \begin{bmatrix} W_1 & W_2 & \cdots \\ W_3 & W_4 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

When necessary, padding is added so that the final representation becomes square. This produces a single large matrix that contains information from many model components. After this combined square matrix is created and normalized, it is converted into a VTP point cloud using the mapping:

$$W_{\text{big}}(i, j) \to (x = i,\ y = j,\ z = W'_{\text{big}}(i, j)).$$

Here, the row index becomes the x-coordinate, the column index becomes the y-coordinate, and the normalized matrix value becomes the z-coordinate. The resulting VTP file is a surface-like three-dimensional object whose height represents the numerical values of the combined matrix.

The advantage of this method is that all selected matrices can be encoded as one object. This makes the input more compact and easier to process through the holographic pipeline, since the encoder receives one large spatial

structure instead of many separate files. It also makes the data more similar to an image-like surface, which is suitable for hologram generation.

However, this approach also has limitations. Since different weight matrices may have different shapes, arranging them into one square layout may require padding or artificial empty regions. These regions do not represent real model parameters, but they still become part of the spatial object. As a result, the holographic encoder may spend part of its capacity encoding empty or artificial structure. Therefore, the layout of the combined square matrix is an important design choice.

7.3 Stacked Matrix Representation

A more model-aware representation is to stack multiple matrices as layers in three-dimensional space. In this strategy, each matrix becomes one surface, and the matrices are placed at different depth levels. If the model contains matrices

$$W = \{W_1, W_2, \ldots, W_n\},$$

then each matrix can be mapped as:

$$W_k(i, j) \to (x = i,\ y = j,\ z = k + \alpha W'_{k,ij}),$$

where k represents the matrix index, and α controls how strongly the normalized matrix values deform each layer.

This representation is closer to the structure of an LLM because the model itself is layered. Each Transformer block contains several matrices, and the full model can be interpreted as a sequence of matrix layers. By stacking matrices, the VTP object becomes a three-dimensional structure rather than one flat surface.

The main advantage of this approach is that it preserves the separation between different matrices. The decoder can reconstruct a layered object, and each reconstructed layer can later be converted back into an approximate matrix. However, this method also introduces challenges. If layers are placed too close together, they may interfere during encoding and decoding. If they are placed too far apart, the object may become too large for efficient holographic representation.

7.4 Hemisphere-Based Representation

Another explored strategy is to map matrix values onto a hemisphere-like surface. In this approach, the matrix is not represented as a flat plane. Instead, its indices are mapped to angular coordinates, and the normalized values influence the radius or height of points on a curved surface.

A simplified version of this mapping can be described as:

$$(i, j, W'_{ij}) \to (r, \theta, \phi),$$

where i and j determine angular position and W'_{ij} affects the radial distance. The resulting coordinates can then be converted into Cartesian form:

$$x = r \sin(\theta) \cos(\phi),$$

$$y = r \sin(\theta) \sin(\phi),$$

$$z = r \cos(\theta).$$

The motivation for this strategy is that holography is naturally related to wavefronts and spatial propagation. A curved representation may distribute information more evenly across a three-dimensional space than a flat surface. It may also reduce the dominance of one viewing direction, because the object is no longer simply a plane viewed from the front.

However, the hemisphere representation is more complex than the flat matrix surface. After reconstruction, the object must be mapped back from curved coordinates into the original matrix structure. This inverse mapping may introduce interpolation errors. The hemisphere approach is useful as an experimental representation, but it requires careful evaluation.

7.5 Comparison of Representation Strategies

The different matrix-to-VTP strategies offer different advantages and disadvantages. The single large square matrix representation allows all selected matrices to be encoded together as one surface-like object, but it may introduce artificial padding and layout boundaries. The stacked representation better reflects the layered structure of neural networks, but it requires careful control of spacing between layers. The hemisphere representation is more physically inspired and may distribute information more naturally in three-dimensional space, but it is harder to invert back into exact matrix form.

Overall, the project treats matrix-to-VTP conversion as an essential part of the compression problem. The holographic encoder does not operate on abstract model parameters directly; it operates on spatial objects. Thus, the way matrices are transformed into geometry strongly affects the final reconstruction. By testing several representation strategies, the project investigates which spatial form is most suitable for holography-inspired model compression.

8 Testing and Results

This section reports the experimental evidence for the proposed compression idea. The goal is not only to show reconstructed images, but also to compare encoded size, original size, and reconstruction accuracy. For this reason, the results are interpreted cautiously: a reconstruction may look recognizable while still being weak under strict numerical metrics.

For testing purposes, the project did not immediately use a full BERT model. Instead, a selected subset of BERT weight matrices was used in order to make the experiments computationally manageable while still preserving the real structure of Transformer-based neural networks. The tested subset contained matrices from two selected BERT Transformer layers, denoted as layer 01 and layer 02. For each layer, six matrices were considered: the query matrix Q, key matrix K, value matrix V, attention output matrix, first feed-forward network matrix $FFN1$, and second feed-forward network matrix $FFN2$.

The attention-related matrices Q, K, V, and attention output each had dimensionality 768×768. The feed-forward matrices had larger rectangular shapes: $FFN1$ had dimensionality 768×3072, while $FFN2$ had dimensionality 3072×768. Therefore, the tested BERT subset contained both square attention matrices and rectangular feed-forward matrices, making it suitable for evaluating different matrix-to-VTP representation strategies.

Matrix Type	Layer Index	Dimensionality
Q	01, 02	768×768
K	01, 02	768×768
V	01, 02	768×768
Attention Output	01, 02	768×768
$FFN1$	01, 02	768×3072
$FFN2$	01, 02	3072×768

Table 1: Selected BERT weight matrices used for testing the matrix-to-VTP and holographic encoding pipeline.

This subset was useful because it represents the main internal components of a BERT Transformer block. The Q, K, and V matrices are used in the self-attention mechanism, the attention output matrix projects the attention result back into the hidden representation space, and the two feed-forward matrices expand and compress the hidden representation through the intermediate dimension. Since BERT-base uses a hidden size of 768 and an intermediate feed-forward size of 3072, these matrices provide a realistic test case for the proposed compression framework.

8.1 Preliminary Reconstruction on Simple Three-Dimensional Objects

Before applying the holography-inspired framework to neural network weight matrices, an earlier version of the pipeline was tested on simple three-dimensional VTP objects. The purpose of this preliminary experiment was to check whether a hologram-like encoding could preserve recognizable spatial information from a scene containing multiple basic geometric components.

In this older pipeline, the input was not yet an LLM weight matrix. Instead, the input was a VTP point cloud containing several simple objects in one shared scene. The codec first normalized the point coordinates. Then, the normalized x and y coordinates were rasterized into a two-dimensional complex object field, while the normalized z-coordinate was encoded as phase. Therefore, the mapping used in this version can be described as a phase-depth surface representation:

$$(x, y, z) \rightarrow F(x, y) = A(x, y)e^{i\phi(z)}.$$

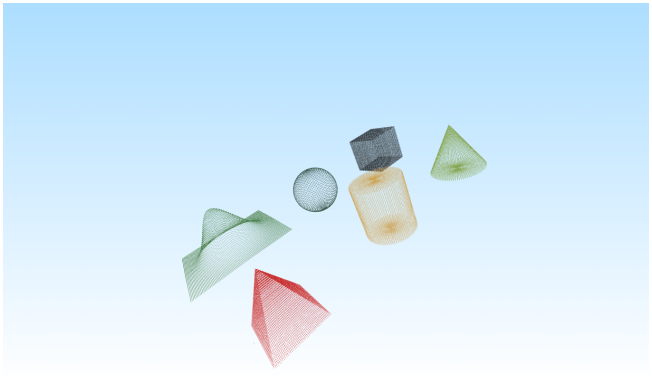
In this representation, the amplitude $A(x, y)$ mainly represents point density or occupancy, while the phase term $\phi(z)$ stores depth information. After constructing this complex object field, a two-dimensional Fourier transform was applied:

$$H = \mathcal{F}\{F(x, y)\},$$

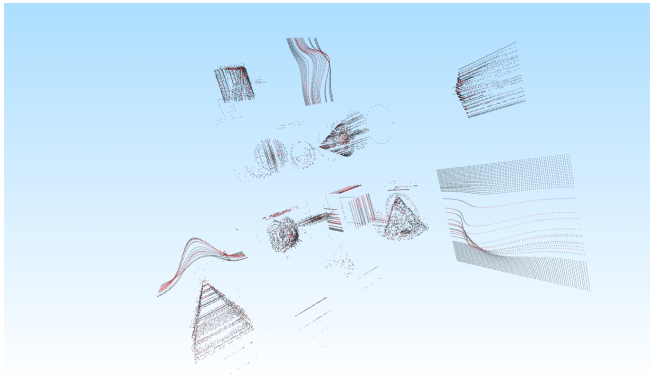
where H is the hologram-like encoded field. During decoding, the inverse Fourier transform was applied to recover the object field. Then, amplitude thresholding was used to select reconstructed points, and the recovered phase was used to estimate the reconstructed depth.

This mapping is closest to a height-field or surface-style representation. It is suitable when each (x, y) position has one main depth value. However, it is less suitable for complex three-dimensional objects where multiple points may share the same projected (x, y) location but have different z-values. In such cases, the phase values may interfere or average together, which can lead to incomplete reconstruction.

Figure 1 shows the result of this preliminary test. The original input file, shown in Figure 1a, contains several complete geometric objects in one scene. The reconstructed output, shown in Figure 1b, does not perfectly reproduce the original scene. Instead, the result appears as decomposed and separated components of the original objects. Some surfaces, outlines, and directional structures remain visible, but the reconstruction is fragmented and noisy.



(a) Original VTP scene containing multiple simple objects.



(b) Reconstructed output from the older phase-depth FFT pipeline.

Figure 1: Preliminary reconstruction of a multi-object VTP scene using the older hologram-like phase-depth FFT pipeline.

The result is important because it demonstrates both the potential and the limitation of the first approach. On one hand, the reconstruction is not random. The decoded output preserves traces of the original geometry and separates the scene into recognizable component-like structures. This suggests that the hologram-like encoding retained some information about the original VTP object.

On the other hand, the reconstruction is not geometrically accurate enough. The original objects are not recovered as clean, complete, and stable three-dimensional shapes. Instead, the output contains partial surfaces, scattered points, and separated fragments. This shows that the phase-depth FFT method can preserve some structural information, but it struggles with complex scenes containing overlapping depths.

This preliminary result motivated the development of the later confidence-volume pipeline. The newer approach uses multi-view encoding, separate amplitude and phase storage, depth-bin propagation, and confidence-based three-dimensional reconstruction. Therefore, this first experiment serves as a useful baseline: it shows that hologram-like encoding can preserve partial geometric structure, while also showing why stronger depth handling and multi-view reconstruction are necessary.

8.2 Full Matrix Roundtrip Reconstruction Experiment

After testing the earlier pipeline on simple three-dimensional objects, the next experiment moved closer to the real goal of the capstone by applying the framework to actual BERT weight matrices. This experiment used three related scripts. The first script performed the full matrix-aware roundtrip pipeline, converting a folder of matrix files

into a grouped VTP point cloud, encoding it into hologram image files, decoding it back into CSV matrices, and reconstructing a grouped VTP file. The second script was designed as a split-and-merge utility for large matrices, especially for feed-forward matrices that are larger than one standard attention matrix. The third script compared the original CSV matrices with the decoded CSV matrices and generated the final similarity report.

The matrix-aware pipeline represents each matrix as a spatial block inside one grouped VTP object. For every matrix element, the column index determines the x-coordinate, the row index determines the y-coordinate, and the matrix value determines the z-coordinate:

$$x = \text{col\_index} \cdot x_{\text{spacing}} + x_{\text{offset}},$$

$$y = \text{row\_index} \cdot y_{\text{spacing}} + y_{\text{offset}},$$

$$z = W_{ij}.$$

The grouped VTP also stores matrix-specific metadata such as the original value, layer index, row index, and column index. This is important because the decoder must not only reconstruct a point cloud visually, but also recover the original matrix structure and save it back as CSV files.

In the encoding stage, the pipeline does not treat the group of matrices as one ordinary image. Instead, it uses a low-frequency FFT tiling strategy. Each matrix is first normalized internally, transformed using a two-dimensional Fourier transform, and then placed into a tile inside a larger spectrum canvas. The final hologram-like field is obtained by applying an inverse Fourier transform to this spectrum canvas. The encoded output is stored through companion image files, including real and imaginary PNG files, together with metadata required for decoding.

During decoding, the process is reversed. The real and imaginary PNG files are loaded to recover the complex field. A Fourier transform is applied to recover the spectrum canvas. Then, the tile corresponding to each matrix is extracted, pasted back into a spectrum of the matrix's original shape, and transformed back into the spatial domain. The reconstructed matrices are then saved as decoded CSV files, and a reconstructed grouped VTP is also generated.

The split-and-merge script was introduced because large rectangular matrices can be harder to reconstruct in this tiling setup. In particular, the feed-forward matrices $FFN1$ and $FFN2$ are much larger than the 768×768 attention matrices. The utility therefore allows large matrices to be divided into smaller blocks, such as 768×768, before hologram encoding. After decoding, the blocks can be merged back into their original matrix shapes. For this BERT-based experiment, $FFN1$ of size 768×3072 can be divided into four 768×768 blocks, and $FFN2$ of size 3072×768 can also be divided into four 768×768 blocks.

For evaluation, the decoded matrices were compared with the original matrices using a separate comparison script. The comparison script computes Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), maximum absolute error, relative Frobenius error, and similarity percentage. The similarity score is defined as:

$$\text{relative error} = \frac{\|W - \hat{W}\|_F}{\|W\|_F},$$

$$\text{similarity} = 100 \cdot (1 - \text{relative error}),$$

where W is the original matrix and $\hat{W}$ is the decoded matrix. This means that the similarity score is based on numerical reconstruction error, not on visual similarity or model accuracy.

The experiment used twelve BERT matrices from two Transformer layers. These included the query, key, value, attention output, first feed-forward, and second feed-forward matrices for layers 01 and 02. The attention-related matrices had shape 768×768, while the feed-forward matrices had shapes 768×3072 and 3072×768.

Matrix Group	**Shape**	**Average MAE**	**Average RMSE**	**Average Similarity**
Attention matrices $Q, K, V,$ Output	768×768	0.204384	0.255446	25.55%
Feed-forward $FFN1$	768×3072	0.204344	0.255457	25.56%
Feed-forward $FFN2$	3072×768	0.204649	0.255790	25.49%
Overall average	Mixed	0.204422	0.255505	25.54%

Table 2: Average reconstruction results for the full matrix roundtrip experiment.

The detailed layer-by-layer results are shown in Table 3. The similarity values are very close across all matrices, ranging from approximately 25.38% to 25.66%. This shows that the reconstruction behavior was stable across different matrix types, but the absolute reconstruction quality was still limited.

Original Matrix	Shape	MAE	RMSE	Similarity
$FFN1\_01$	768×3072	0.204329	0.255413	25.57%
$FFN1\_02$	768×3072	0.204360	0.255501	25.56%
$FFN2\_01$	3072×768	0.204643	0.255785	25.48%
$FFN2\_02$	3072×768	0.204655	0.255794	25.49%
K_01	768×768	0.204658	0.255758	25.53%
K_02	768×768	0.204737	0.255848	25.38%
Q_01	768×768	0.204679	0.255823	25.57%
Q_02	768×768	0.203974	0.254852	25.65%
V_01	768×768	0.203656	0.254556	25.66%
V_02	768×768	0.204610	0.255732	25.47%
Attention Output 01	768×768	0.204277	0.255359	25.58%
Attention Output 02	768×768	0.204483	0.255641	25.53%

Table 3: Layer-by-layer reconstruction results from the decoded CSV similarity report.

The result of this experiment is important because it demonstrates that the full reconstruction pipeline was technically successful: matrix files were converted into a grouped VTP representation, encoded into hologram-like files, decoded back into CSV matrices, and compared numerically against the originals. However, the numerical similarity remained low, at approximately 25.54% on average. This indicates that although the framework can perform a complete roundtrip reconstruction, the decoded matrices are still far from the original BERT matrices in strict numerical terms.

The main reason for this limitation is that the FFT tiling approach preserves only part of each matrix's frequency information. When a matrix spectrum is cropped into a limited tile, high-frequency details are lost. This is especially problematic for neural network weights, where small numerical differences may matter even if the reconstructed matrix visually appears structurally similar. Therefore, this experiment showed that a complete matrix reconstruction workflow is possible, but stronger encoding and decoding methods are necessary before the approach can be considered effective for model compression.

Overall, this experiment represents a transition from simple geometric reconstruction to actual neural network parameter reconstruction. Compared with the simple-object test, this experiment is more directly connected to the capstone goal because it reconstructs BERT matrix data rather than synthetic VTP geometry. At the same time, the low similarity score motivates later improvements, such as better preservation of frequency components, larger hologram sizes, block-based reconstruction, phase-aware encoding, and confidence-based decoding.

8.3 Cube Reconstruction Using the Confidence-Volume Decoder

The third experiment used a newer version of the VTP holography pipeline, called the confidence-volume decoder. Unlike the earlier FFT-based phase-depth codec, this tool is designed specifically for more reliable three-dimensional reconstruction. Its purpose is to encode a VTP point cloud into a hologram-like package and then reconstruct the object using multi-view evidence.

The pipeline follows the structure:

$$\text{initial VTP} \rightarrow \text{amplitude} + \text{phase} + \text{metadata} \rightarrow \text{confidence-volume decoding} \rightarrow \text{reconstructed VTP}.$$

In the encoding stage, the input VTP point cloud is first normalized into a common coordinate system. Then the object is observed from multiple views. In the cube experiment, the multi-view setup is important because a cube cannot be fully represented from only one direction. For each view, the object is divided into depth bins. Each depth slice is projected onto a two-dimensional complex plane and propagated using an angular-spectrum-style wave propagation method. The result is a complex hologram field for each view. This field is stored using two images: an amplitude PNG and a phase PNG. A metadata file stores the normalization parameters, view rotations, depth-bin information, hologram size, and other reconstruction settings.

The main improvement of this tool is in the decoding stage. Instead of directly taking the brightest reconstructed pixels as final points, the decoder back-propagates each view through the depth bins and collects candidate evidence in a shared three-dimensional confidence volume. A voxel is treated as more reliable when it receives support from multiple views. The decoder can also remove weak evidence and isolated components. Therefore, the reconstruction is based on multi-view agreement rather than one projection alone.

This experiment was applied to a cube object, `cube_average.vtp`. Figure 2 compares the original cube and the reconstructed result. Visually, the reconstruction preserves the general cube-like structure. The reconstructed object is not a perfect copy of the original cube, but it shows that the newer confidence-volume decoder is much more suitable for closed three-dimensional objects than the older phase-depth FFT method.

(a) Original cube object.

(b) Reconstructed cube using the confidence-volume decoder.

Figure 2: Cube reconstruction using the newer Gabor-style confidence-volume decoder.

The numerical results are summarized in Table 4. The original file contained 13,256 points, while the reconstructed file contained 18,000 points. The encoded hologram package had a size of 384,468 bytes, compared with the original VTP size of 674,302 bytes. This gives a compression ratio of approximately 1.75×. Thus, the method achieved actual compression, although the compression level was moderate rather than extreme.

Metric	**Value**
Original points	13,256
Reconstructed points	18,000
Original VTP size	674,302 bytes
Encoded hologram size	384,468 bytes
Compression ratio	1.7539×
Original-to-reconstructed RMSE	0.056166
Reconstructed-to-original RMSE	0.119892
Symmetric Hausdorff 95	0.244163
Symmetric Hausdorff max	0.819432
F1 at one estimated spacing	44.87%
F1 at two estimated spacings	78.91%
Voxel IoU at resolution 128	1.45%
Outside original bounding box	37.81%

Table 4: Strict reconstruction metrics for the cube confidence-volume experiment.

The nearest-neighbor distances show that the reconstruction captured the cube structure partially but not perfectly. The original-to-reconstructed RMSE was 0.056166, while the estimated original point spacing was 0.042553. This means that many original cube points had a reconstructed point reasonably nearby, although not exactly at the same location. The reconstructed-to-original RMSE was higher, 0.119892, which suggests that the reconstructed point cloud contained extra or displaced points that did not lie as close to the original cube surface.

The precision, recall, and F1 results give a more balanced interpretation. At a strict threshold of one estimated point spacing, the F1 score was 44.87%. At two spacings, the F1 score increased to 78.91%. This suggests that the reconstructed cube is geometrically close at a broader tolerance, even though exact point-level reconstruction remains difficult. At larger thresholds, the F1 score becomes very high, reaching 95.94% and 99.63%, which means that the reconstructed object remains generally near the original cube region.

However, the voxel IoU values are low, especially at high resolution. The voxel IoU was only 1.45% at resolution 128 and 0.00% at resolution 256. This indicates that exact voxel-level overlap between the original and reconstructed point clouds is weak. In other words, the reconstructed object has a similar general shape, but its points do not occupy the exact same fine-grid cells as the original object. This is expected for a noisy holographic reconstruction, but it also shows that the method is not yet precise enough for exact geometric recovery.

The cube-surface metrics also show an important limitation. About 37.81% of reconstructed points were outside the original bounding box. This means that the decoder still produces a significant amount of off-surface or outside-region evidence. At the same time, the average distance to the nearest cube face was 0.075586, which suggests that many reconstructed points still remain close to the cube surface. Therefore, the reconstruction is not random, but it is also not clean enough to be considered a highly accurate cube recovery.

Overall, this experiment is a meaningful improvement over the older simple-object reconstruction. The confidence-volume decoder reconstructs a recognizable cube-like object and achieves moderate compression. The result shows that multi-view amplitude-and-phase encoding combined with confidence-volume decoding is a better direction for

three-dimensional reconstruction. At the same time, the strict metrics show that the method still has limitations: the reconstruction contains extra points, has low fine-scale voxel overlap, and places a noticeable fraction of points outside the original bounding box. This experiment supports the feasibility of the approach while also showing that further refinement is needed before applying the same idea to sensitive numerical data such as LLM weight matrices.

9 Future Work

The current project focuses on designing and testing a holography-inspired encoding and decoding framework through algorithmic reconstruction. The experiments show that geometric and matrix-based information can be encoded into hologram-like representations and reconstructed to some extent. However, strict numerical accuracy remains limited, especially for neural-network matrices. The most important future direction is therefore to support or replace the hand-designed decoder with deep learning models trained specifically for hologram reconstruction.

A promising direction is to train image-to-image or image-to-volume neural networks, such as U-Net-style models, to reconstruct the original object or matrix representation from the encoded hologram [17]. U-Net is relevant because its encoder–decoder structure with skip connections can preserve both global structure and local detail. In this project, the input could be a multi-channel hologram representation, with amplitude and phase stored as separate channels. The target output could be an original matrix, a reconstructed depth map, a point-cloud occupancy grid, or a voxelized version of the original VTP object.

The future learned-decoding pipeline can be written as:

$$W \to P \to H \to \mathcal{N}_\theta(H) \to \hat{W},$$

where W is the original matrix or group of matrices, P is the spatial representation, H is the encoded hologram package, $\mathcal{N}_\theta$ is a trained neural decoder, and $\hat{W}$ is the reconstructed matrix output.

This approach would change the decoder from a fixed reconstruction rule into a learned inverse mapping. The current decoder relies on back-propagation, thresholding, confidence-volume voting, and connected-component filtering. These operations are useful, but they may not fully capture the relationship between the encoded hologram and the original object. A neural decoder could learn this relationship from many examples of original objects and their encoded holograms:

$$(P_1, H_1), (P_2, H_2), \ldots, (P_n, H_n),$$

or, in the matrix case:

$$(W_1, H_1), (W_2, H_2), \ldots, (W_n, H_n).$$

Possible loss functions include Mean Squared Error (MSE), Mean Absolute Error (MAE), cosine similarity loss, structural similarity loss, Chamfer distance for point clouds, and voxel IoU-based losses for three-dimensional reconstructions. For LLM weight matrices, the loss function should also include matrix-level similarity measures such as relative Frobenius error:

$$\mathcal{L}_F = \frac{\|W - \hat{W}\|_F}{\|W\|_F}.$$

A future version of the project should also evaluate functional model behavior. Numerical similarity alone is not enough for LLM compression. After reconstructing weights, the reconstructed matrices should be inserted back into a model and tested on downstream tasks. This would answer the more important question: not only whether the weights are numerically close, but whether the reconstructed model behaves similarly to the original model.

Several architectures could be explored. A two-dimensional U-Net could be used when the hologram is treated as a multi-channel image. A three-dimensional U-Net could be used if the target output is a voxelized object or a multi-layer matrix volume. Autoencoders could learn compact latent representations of hologram packages. Physics-informed neural networks could combine wave-propagation operations with learnable layers, allowing the system to use both holographic principles and data-driven reconstruction.

Training should begin with synthetic data before moving to real model weights. A large dataset of cubes, spheres, bell-shaped functions, surfaces, artificial matrix patterns, and random structured objects could be generated automatically. The model could first learn general reconstruction behavior from these controlled examples. After that, it could be fine-tuned on actual neural-network weight matrices, such as BERT attention and feed-forward layers.

The main limitation of this future direction is computational resources. Training a neural reconstruction model requires many encoded holograms with corresponding ground-truth objects or matrices. It also requires GPU

memory, storage, and time for repeated training and evaluation. For small experiments, a U-Net could be trained on lower-resolution holograms and smaller matrix blocks. For realistic LLM-scale compression, however, the dataset and hardware requirements would be much larger. Thus, deep learning is not only a methodological improvement but also a resource-dependent extension.

Future work should also explore block-based learning. Large matrices such as 768×3072 or 3072×768 can be split into smaller blocks, reconstructed independently, and then merged back together. This would reduce memory requirements and make neural decoding more practical. A learned decoder could be trained on fixed-size blocks, such as 768×768 or smaller patches, instead of reconstructing an entire model at once.

Another improvement is residual learning. The current algorithmic decoder could first produce an approximate reconstruction $\hat{W}_0$. A neural network would then learn the residual error:

$$R = W - \hat{W}_0.$$

The final reconstruction would be:

$$\hat{W} = \hat{W}_0 + \mathcal{N}_\theta(H).$$

This residual strategy may be easier than full reconstruction because the model only needs to correct systematic decoder errors. Overall, the future direction is to move from purely algorithmic decoding toward learned hologram reconstruction. The current framework already generates encoded hologram packages, reconstructed outputs, and strict metrics, which makes it suitable for producing training data and evaluating future neural decoders.

10 Conclusion

This capstone project explored whether holography-inspired encoding and decoding can be used as a possible compression mechanism for large language model weight matrices. The main idea was to treat neural-network weights not only as numerical tables, but also as spatial structures that can be transformed into point clouds, surfaces, volumes, or hologram-like representations. In this sense, the project focused on holography as an alternative encoding domain: information is converted into amplitude-and-phase hologram packages and later decoded back into an approximate geometric or matrix representation.

The implemented framework successfully demonstrates the full experimental pipeline: matrix or object normalization, VTP point-cloud construction, hologram-like encoding, amplitude and phase storage, confidence-volume decoding, reconstruction into VTP format, and strict evaluation through size and accuracy metrics. This shows that the project is not only theoretical, but also supported by a working prototype that can encode, decode, and measure reconstruction quality.

The strongest result was obtained in the cube reconstruction experiment. The original cube file had $13,256$ points and a size of $674,302$ bytes, while the encoded hologram package had a size of $384,468$ bytes. This produced a compression ratio of approximately $1.7539\times$. The reconstruction preserved a recognizable cube-like structure, with an original-to-reconstructed RMSE of 0.056166 and an F1 score of 78.91% at two estimated point spacings. These results suggest that the method can preserve meaningful geometric information while reducing storage size.

At the same time, the strict metrics show that the method is not yet accurate enough for reliable LLM compression. The voxel IoU remained low, and a significant fraction of reconstructed points appeared outside the original cube bounding box. The matrix reconstruction experiments also showed that visual or structural similarity does not automatically imply exact numerical recovery. This is especially important for LLM weights, where small numerical errors may affect model behavior. Therefore, the current system should be interpreted as an exploratory proof of concept rather than a practical replacement for existing compression methods such as quantization, pruning, distillation, or low-rank approximation.

Overall, the project supports the possibility of using holography-inspired representations for compression research, but it also identifies the main limitation: decoding quality. The encoded hologram package can store compressed structural information, but the current algorithmic decoder cannot yet recover that information with sufficient numerical precision. Future work should therefore focus on learned reconstruction methods, especially U-Net-style neural decoders trained on pairs of original objects or matrices and their encoded holograms. Such models may be able to recover details that are lost by thresholding, back-propagation, and confidence-volume voting alone.

The main contribution of this capstone is the design and evaluation of a complete holographic encoding–decoding framework for spatial representations of model weights. The experiments show both the promise and the difficulty of this direction: hologram-like encoding can reduce storage and preserve recognizable structure, but accurate reconstruction remains the central challenge. For this reason, holographic compression should be considered a future

research direction for LLM compression, especially if combined with deep learning, better phase-aware encoding, block-based matrix reconstruction, and downstream model-performance evaluation.

References

[1] Gabor, D. (1948). A new microscopic principle. *Nature, 161*, 777–778. https://doi.org/10.1038/161777a0

[2] Goodman, J. W. (2005). *Introduction to Fourier Optics* (3rd ed.). Roberts and Company Publishers.

[3] Joint Photographic Experts Group. (2024). *White Paper on JPEG Pleno Holography*. JPEG. https://ds.jpeg.org/whitepapers/wg1n101066-106-COM-White_Paper_on_JPEG_Pleno_Holography.pdf

[4] Brady, D. J., Choi, K., Marks, D. L., Horisaki, R., & Lim, S. (2009). Compressive holography. *Optics Express, 17*(15), 13040–13049. https://doi.org/10.1364/OE.17.013040

[5] Birdi, J., Sunaina, Butola, M., & Khare, K. (2020). True 3D reconstruction in digital holography. *Journal of Physics: Photonics, 2*(4), 044004. https://doi.org/10.1088/2515-7647/aba5a3

[6] Shi, L., Webb, R., Xiao, L., Kim, C., & Jang, C. (2022). Neural compression for hologram images and videos. *Optics Letters, 47*(22), 6013–6016. https://doi.org/10.1364/OL.472769

[7] Shimobaba, T., Blinder, D., Birnbaum, T., Hoshi, I., Shiomi, H., Schelkens, P., & Ito, T. (2022). Deep-learning computational holography: A review. *Frontiers in Photonics, 3*, 854391. https://doi.org/10.3389/fphot.2022.854391

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/1706.03762

[9] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

[10] Zhu, X., Li, J., Liu, Y., Ma, C., & Wang, W. (2024). A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics, 12*, 1556–1577. https://doi.org/10.1162/tacl_a_00704

[11] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*. https://arxiv.org/abs/2210.17323

[12] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*. https://arxiv.org/abs/2211.10438

[13] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2023). AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*. https://arxiv.org/abs/2306.00978

[14] Frantar, E., & Alistarh, D. (2023). SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning*. https://arxiv.org/abs/2301.00774

[15] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*. https://arxiv.org/abs/1910.01108

[16] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*. https://arxiv.org/abs/2106.09685

[17] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015* (pp. 234–241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28

[18] Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.

[19] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[20] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., & SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

[21] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

[22] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

[23] Schroeder, W., Martin, K., & Lorensen, B. (2006). *The Visualization Toolkit: An Object-Oriented Approach to 3D Graphics* (4th ed.). Kitware.

[24] Pillow Contributors. (n.d.). *Pillow: The Friendly PIL Fork*. Pillow documentation. https://pillow.readthedocs.io/